LESSON NAME

Intro

Prerequisites

- FIXME
- …
- …

20 min filename

CuPy

Questions

- How could I make my Python code to run on a GPU
- How do I copy data to the GPU memory

Objectives

- Learn the basics of CuPy
- Be able to find out if a variable is stored in the CPU or GPU memory
- Be able to copy data from host to device memory and vice versa
- Be able to re-write a simple NumPy/SciPy function using CuPy to run on the GPUs

Introduction to CuPy

Another excellent tool for writing Python code to run on GPUs is CuPy. CuPy is a NumPy/SciPy-compatible array library for GPU-accelerated computing with Python, which implements most of the NumPy/SciPy operations and acts as a drop-in replacement to run existing NumPy/SciPy code on both NVIDIA CUDA or AMD ROCm platforms. By design, the CuPy interface is as close as possible to NumPy/SciPy, making code porting much easier.

Basics of CuPy

Skip to content

re is identical to that of NumPy. A list of NumPy/SciPy APIs and its Py implementations is summarised here:

Complete Comparison of NumPy and SciPy to CuPy functions.

In short, CuPy provides N-dimensional array (ndarray), sparse matrices, and the associated routines for GPU devices, most having the same API as NumPy/SciPy.

Let us take a look at the following code snippet which calculates the L2-norm of an array. Note how simple it is to run on a GPU device using CuPy, i.e. essentially by changing np to cp.

NumPy

```
import numpy as np
x_cpu = np.array([1, 2, 3])
l2_cpu = np.linalg.norm(x_cpu)
```

CuPy

```
import cupy as cp
x_gpu = cp.array([1 ,2 ,3])
l2_gpu = cp.linalg.norm(x_gpu)
```

> **Warning**
>
> Do not change the import line in the code to something like
>
> `import cupy as np`
>
> which can cause problems if you need to use NumPy code and not CuPy code.

Conversion to/from NumPy arrays

Although cupy.ndarray is the CuPy counterpart of NumPy numpy.ndarray, the main difference is that cupy.ndarray resides on the `current device`, and they are not implicitly convertible to each other. When you need to manipulate CPU and GPU arrays, an explicit data transfer may be required to move them to the same location – either CPU or GPU. For this purpose, CuPy implements the following methods:

- To convert numpy.ndarray to cupy.ndarray, use cupy.array() or cupy.asarray()
- To convert cupy.ndarray to numpy.ndarray, use cupy.asnumpy() or cupy.ndarray.get()

These methods can accept arbitrary input, meaning that they can be applied to any data that is located on either the host or device.

Here is an example that demonstrates the use of both methods:

Skip to content

```
>>> import numpy as np
>>> import cupy as cp
>>>
>>> x_cpu = np.array([1, 2, 3]) # allocating array x on cpu
>>> y_cpu = np.array([4, 5, 6]) # allocating array y on cpu
>>> x_cpu + y_cpu # add x and y
array([5, 7, 9])
>>>
>>> x_gpu = cp.asarray(x_cpu) # move x to gpu
>>> x_gpu + y_cpu # now it should fail
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "cupy/_core/core.pyx", line 1375, in cupy._core.core._ndarray_base.__add__
  File "cupy/_core/core.pyx", line 1799, in cupy._core.core._ndarray_base.__array_ufunc
  File "cupy/_core/_kernel.pyx", line 1285, in cupy._core._kernel.ufunc.__call__
  File "cupy/_core/_kernel.pyx", line 159, in cupy._core._kernel._preprocess_args
  File "cupy/_core/_kernel.pyx", line 145, in cupy._core._kernel._preprocess_arg
TypeError: Unsupported type <class 'numpy.ndarray'>
>>>
>>> cp.asnumpy(x_gpu) + y_cpu
array([5, 7, 9])
>>> cp.asnumpy(x_gpu) + cp.asnumpy(y_cpu)
array([5, 7, 9])
>>> x_gpu + cp.asarray(y_cpu)
array([5, 7, 9])
>>> cp.asarray(x_gpu) + cp.asarray(y_cpu)
array([5, 7, 9])
```

> **Note**
>
> Converting between cupy.ndarray and numpy.ndarray incurs data transfer between the host (CPU) device and the GPU device, which is costly in terms of performance.

Current Device

CuPy introduces the concept of a `current device`, which represents the default GPU device on which the allocation, manipulation, calculation, etc., of arrays take place. cupy.ndarray.device attribute can be used to determine the device allocated to a CuPy array. By default, ID of the current device is 0.

```
>>> import cupy as cp
>>> x_gpu = cp.array([1, 2, 3, 4, 5])
>>> x_gpu.device
```

Skip to content

To obtain the total number of accessible devices, one can utilize the `getDeviceCount` function:

```
>>> import cupy as cp
>>> cp.cuda.runtime.getDeviceCount()
1
```

To switch to another GPU device, use the `Device` context manager. For example, the following code snippet creates an array on GPU 1:

```
>>> import cupy as cp
>>> with cp.cuda.Device(1):
       x_gpu1 = cp.array([1, 2, 3, 4, 5])
>>> print("x_gpu1 is on device:" x_gpu1.device)
```

All CuPy operations (except for multi-GPU features and device-to-device copy) are performed on the currently active device.

> **Note**
>
> The device will be called <CUDA Device 0> even if you are on AMD GPUs.
>
> In general, CuPy functions expect that the data array is on the current device. Passing an array stored on a non-current device may work depending on the hardware configuration but is generally discouraged as it may not be performant.

Exercises: Matrix Multiplication

> **Exercise : Matrix Multiplication**
>
> The first example is a simple matrix multiplication in single precision (float32). The arrays are created with random values in the range of -1.0 to 1.0. Convert the NumPy code to run on GPU using CuPy.

```
import math
import numpy as np

A = np.random.uniform(low=-1., high=1., size=(64,64)).astype(np.float32)
B = np.random.uniform(low=-1., high=1., size=(64,64)).astype(np.float32)
C = np.matmul(A,B)
```

> **Solution**

```
import math
import cupy as cp

A = cp.random.uniform(low=-1., high=1., size=(64,64)).astype(cp.float32)
B = cp.random.uniform(low=-1., high=1., size=(64,64)).astype(cp.float32)
C = np.matmul(A,B)
```

Skip to content

> Notice in this snippet of code that the variable C remains on the GPU. You have to copy it back to the CPU explicitly if needed. Otherwise all the data on the GPU is wiped once the code ends.

Exercises: moving data from GPU to CPU

Exercise : moving data from GPU to CPU

The code snippet simply computes a singular value decomposition (SVD) of a matrix. In this case, the matrix is a single-precision 64x64 matrix of random values. First re-write the code using CuPy for GPU enabling. Second, adding a few lines to copy variable u back to CPU and print objects' type using `type()` function.

```
import numpy as np

A = np.random.uniform(low=-1., high=1., size=(64, 64)).astype(np.float32)
u, s, v = np.linalg.svd(A)
```

Solution

```
import cupy as cp

A = cp.random.uniform(low=-1., high=1., size=(64, 64)).astype(cp.float32)
u_gpu, s_gpu, v_gpu = cp.linalg.svd(A)
print("type(u_gpu) = ",type(u_gpu))
u_cpu = cp.asnumpy(u_gpu)
print("type(u_cpu) = ",type(u_cpu))
```

Exercises: CuPy vs Numpy/SciPy

CuPy vs Numpy/SciPy

Although the CuPy team focuses on providing a complete NumPy/SciPy API coverage to become a full drop-in replacement, some important differences between CuPy and NumPy should be noted, one should keep these differences in mind when porting NumPy code to CuPy.

Here are various examples illustrating the differences

Skip to content

When CuPy is different from NumPy/SciPy

Cast behavior from float to integer

Some casting behaviors from float to integer are not defined in C++ specification. The casting from a negative float to unsigned integer and infinity to integer is one of such examples. The behavior of NumPy depends on your CPU architecture. This is the result on an Intel CPU:

```
>>> np.array([-1], dtype=np.float32).astype(np.uint32)
array([4294967295], dtype=uint32)

>>> cp.array([-1], dtype=np.float32).astype(np.uint32)
array([0], dtype=uint32)
```

```
>>> np.array([float('inf')], dtype=np.float32).astype(np.int32)
array([-2147483648], dtype=int32)

>>> cp.array([float('inf')], dtype=np.float32).astype(np.int32)
array([2147483647], dtype=int32)
```

Random methods support dtype argument

NumPy's random value generator does not support a dtype argument and instead always returns a float64 value. While in CuPy, both float32 and float64 are supported because of cuRAND.

```
>>> np.random.randn(dtype=np.float32)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: randn() got an unexpected keyword argument 'dtype'

>>> cp.random.randn(dtype=np.float32)
array(1.3591791, dtype=float32)
```

Out-of-bounds indices

CuPy handles out-of-bounds indices differently by default from NumPy when using integer array indexing. NumPy handles them by raising an error, but CuPy wraps around them.

```
>>> x = np.array([0, 1, 2])
>>> x[[1, 3]] = 10
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: index 3 is out of bounds for axis 0 with size 3

>>> x = cp.array([0, 1, 2])
>>> x[[1, 3]]
array([1, 0])
>>> x[[1, 3]] = 10
>>> x
array([10, 10,  2])
```

Skip to content

in indices

CuPy's `__setitem__` behaves differently from NumPy when integer arrays reference the same location multiple times. NumPy stores the value corresponding to the last element among elements referencing duplicate locations. In CuPy, the value that is actually stored is undefined.

```
>>> a = np.zeros((2,))
>>> i = np.arange(10000) % 2
>>> v = np.arange(10000).astype(np.float32)
>>> a[i] = v
>>> a
array([9998., 9999.])

>>> a = cp.zeros((2,))
>>> i = cp.arange(10000) % 2
>>> v = cp.arange(10000).astype(cp.float32)
>>> a[i] = v
>>> a
array([4592., 4593.])
```

Zero-dimensional array

Reduction methods

NumPy's reduction functions (e.g. numpy.sum()) return scalar values (e.g. numpy.float32). However CuPy counterparts return zero-dimensional cupy.ndarray. That is because CuPy scalar values (e.g. cupy.float32) are aliases of NumPy scalar values and are allocated in CPU memory. If these types were returned, it would be required to synchronize between GPU and CPU. If you want to use scalar values, cast the returned arrays explicitly.

```
>>> type(np.sum(np.arange(3))) == np.int64
True

>>> type(cp.sum(cp.arange(3))) == cp.ndarray
True
```

Type promotion

CuPy automatically promotes dtypes of cupy.ndarray in a function with two or more operands, the result dtype is determined by the dtypes of the inputs. This is different from NumPy's rule on type promotion, when operands contain zero-dimensional arrays. Zero-dimensional numpy.ndarray are treated as if they were scalar values if they appear in operands of NumPy's function, This may affect the dtype of its output, depending on the values of the "scalar" inputs.

```
>>> (np.array(3, dtype=np.int32) * np.array([1., 2.], dtype=np.float32)).dtype
dtype('float32')

>>> (np.array(300000, dtype=np.int32) * np.array([1., 2.], dtype=np.float32)).dtype
dtype('float64')

>>> (cp.array(3, dtype=np.int32) * cp.array([1., 2.], dtype=np.float32)).dtype
dtype('float64')

……"### not working
    dtype=np.int32) * np.array([1., 2.], dtype=np.float32)).dtype
```

Skip to content

Matrix type (numpy.matrix)

SciPy returns numpy.matrix (a subclass of numpy.ndarray) when dense matrices are computed from sparse matrices (e.g., coo_matrix + ndarray). However, CuPy returns cupy.ndarray for such operations.

There is no plan to provide numpy.matrix equivalent in CuPy. This is because the use of numpy.matrix is no longer recommended since NumPy 1.15.

Data types

Data type of CuPy arrays cannot be non-numeric like strings or objects.

Universal Functions only work with CuPy array or scalar

Unlike NumPy, Universal Functions in CuPy only work with CuPy array or scalar. They do not accept other objects (e.g., lists or numpy.ndarray).

```
>>> np.power([np.arange(5)], 2)
array([[ 0,  1,  4,  9, 16]])

>>> cp.power([cp.arange(5)], 2)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "cupy/_core/_kernel.pyx", line 1285, in cupy._core._kernel.ufunc.__call__
  File "cupy/_core/_kernel.pyx", line 159, in cupy._core._kernel._preprocess_args
  File "cupy/_core/_kernel.pyx", line 145, in cupy._core._kernel._preprocess_arg
TypeError: Unsupported type <class 'list'>
```

Random seed arrays are hashed to scalars

Like Numpy, CuPy's RandomState objects accept seeds either as numbers or as full numpy arrays.

However, unlike Numpy, array seeds will be hashed down to a single number and so may not communicate as much entropy to the underlying random number generator.

```
>>> seed = np.array([1, 2, 3, 4, 5])
>>> rs = cp.random.RandomState(seed=seed)
```

NaN (not-a-number) handling

By default CuPy's reduction functions (e.g., cupy.sum()) handle NaNs in complex numbers differently from NumPy's counterparts:

```
################## not working
>>> a = [0.5 + 3.7j, complex(0.7, np.nan), complex(np.nan, -3.9), complex(np.nan, np.nan)]
>>> a
[(0.5+3.7j), (0.7+nanj), (nan-3.9j), (nan+nanj)]

>>> a_np = np.asarray(a)
>>> print(a_np.max(), a_np.min())
(0.7+nanj) (0.7+nanj)

>>> a_cp = cp.array(a_np)
>>> print(a_cp.max(), a_cp.min())
(0.7+nanj) (0.7+nanj)
```

Skip to content

The reason is that internally the reduction is performed in a strided fashion, thus it does not ensure a proper comparison order and cannot follow NumPy's rule to always propagate the first-encountered NaN. Note that this difference does not apply when CUB is enabled (which is the default for CuPy v11 or later.)

Contiguity / Strides

To provide the best performance, the contiguity of a resulting ndarray is not guaranteed to match with that of NumPy's output.

```
>>> a = np.array([[1, 2], [3, 4]], order='F')
>>> a
array([[1, 2],
       [3, 4]])
>>> print((a + a).flags.f_contiguous)
True

>>> a = cp.array([[1, 2], [3, 4]], order='F')
>>> a
array([[1, 2],
       [3, 4]])
>>> print((a + a).flags.f_contiguous)
False
```

Interoperability

CuPy implements standard APIs for data exchange and interoperability, which means it can be used in conjunction with any other libraries supporting the standard. For example, NumPy, Numba, PyTorch, TensorFlow, MPI4Py among others can be directly operated on CuPy arrays.

NumPy

CuPy implements `__array_ufunc__` interface (see NEP 13), `__array_function__` interface (see NEP 18), and other Python Array API Standard.

Note that the return type of these operations is still consistent with the initial type.

```
>>> import cupy as cp
>>> import numpy as np
>>> dev_arr = cp.random.randn(1, 2, 3, 4).astype(cp.float32)
>>> result = np.sum(dev_arr)
>>> print(type(result))
<class 'cupy.ndarray'>
```

Skip to content

> **Note**
>
> `__array_ufunc__` feature requires NumPy 1.13 or later
>
> `__array_function__` feature requires NumPy 1.16 or later. As of NumPy 1.17, `__array_function__` is enabled by default
>
> NEP 13 — A mechanism for overriding Ufuncs
>
> NEP 18 — A dispatch mechanism for NumPy's high level array functions

Numba

CuPy implements `__cuda_array_interface__` which is compatible with Numba v0.39.0 or later (see CUDA Array Interface for details). It means one can pass CuPy arrays to kernels JITed with Numba.

```
import cupy as cp
from numba import cuda

@cuda.jit
def add(x, y, out):
        start = cuda.grid(1)
        stride = cuda.gridsize(1)
        for i in range(start, x.shape[0], stride):
                out[i] = x[i] + y[i]

a = cp.arange(10)
b = a * 2

out = cp.zeros_like(a)
print(out)  # => [0 0 0 0 0 0 0 0 0 0]

add[1, 32](a, b, out)
print(out)  # => [ 0  3  6  9 12 15 18 21 24 27]
```

In addition, cupy.asarray() supports zero-copy conversion from Numba CUDA array to CuPy array.

Skip to content

```
>>> import numpy as np
>>> import cupy as cp
>>> import numba
>>> x = np.arange(10)
>>> type(x)
<class 'numpy.ndarray'>
>>> x_numba = numba.cuda.to_device(x)
>>> type(x_numba)
<class 'numba.cuda.cudadrv.devicearray.DeviceNDArray'>
>>> x_cupy = cp.asarray(x_numba)
>>> type(x_cupy)
<class 'cupy.ndarray'>
```

CPU/GPU agnostic code

Once beginning porting code to the GPU, one has to consider how to handle creating data on either the CPU or GPU. CuPy's compatibility with NumPy/SciPy makes it possible to write CPU/GPU agnostic code. For this purpose, CuPy implements the cupy.get_array_module() function that returns a reference to cupy if any of its arguments resides on a GPU and numpy otherwise.

Here is an example of a CPU/GPU agnostic function

```
>>> import numpy as np
>>> import cupy as cp
>>> # define a simple function: f(x)=x+1
>>> def addone(x):
...     xp = cp.get_array_module(x) # Returns cupy if any array is on the GPU, otherwis
...     print("Using:", xp.__name__)
...     return x+1
>>> # create an array and copy it to GPU
>>> a_cpu = np.arange(0, 20, 2)
>>> a_gpu = cp.asarray(a_cpu)
>>> # GPU/CPU agnostic code also works with CuPy
>>> print(addone(a_cpu))
Using: numpy
[ 1  3  5  7  9 11 13 15 17 19]
>>> print(addone(a_gpu))
Using: cupy
[ 1  3  5  7  9 11 13 15 17 19]
```

User-Defined Kernels

Sometimes you need a specific GPU function or routine that is not provided by an existing library

Skip to content

In such a situation, you need to write a "custom kernel", i.e. a user-defined GPU kernel. Custom kernels written with CuPy only require a small snippet of code and CuPy automatically wraps and compiles it. Compiled binaries are then cached and reused in subsequent runs.

CuPy provides three templates of user-defined kernels:

- cupy.ElementwiseKernel: User-defined elementwise kernel
- cupy.ReductionKernel: User-defined reduction kernel
- cupy.RawKernel: User-defined CUDA/HIP kernel

ElementwiseKernel

The element-wise kernel focuses on kernels that operate on an element-wise basis. An element-wise kernel has four components:

- input argument list
- output argument list
- function code
- kernel name

The argument lists consist of comma-separated argument definitions. Each argument definition consists of a type specifier and an argument name. Names of NumPy data types can be used as type specifiers.

```
>>> my_kernel = cp.ElementwiseKernel(
...     'float32 x, float32 y',  # input arg list
...     'float32 z',             # output arg list
...     'z = (x - y) * (x - y)', # function
...     'my_kernel')             # kernel name
```

In the first line, the object instantiation is named `kernel`. The next line has the variables to be used as input (x and y) and output (z). These variables can be typed with NumPy data types, as shown. The function code then follows. The last line states the kernel name, which is `my_kernel`, in this case.

The above kernel can be called on either scalars or arrays since the ElementwiseKernel class does the indexing with broadcasting automatically:

Skip to content

```
>>> import cupy as cp
>>> # user-defined kernel
>>> my_kernel = cp.ElementwiseKernel(
...     'float32 x, float32 y',
...     'float32 z',
...     'z = (x - y) * (x - y)',
...     'my_kernel')
>>> # allocating arrays x and y
>>> x = cp.arange(10, dtype=np.float32).reshape(2, 5)
>>> y = cp.arange(5, dtype=np.float32)
>>> # launch the kernel
>>> my_kernel(x,y)
array([[ 0.,  0.,  0.,  0.,  0.],
       [25., 25., 25., 25., 25.]], dtype=float32)
>>> my_kernel(x,5)
array([[25., 16.,  9.,  4.,  1.],
       [ 0.,  1.,  4.,  9., 16.]], dtype=float32)
```

Sometimes it would be nice to create a generic kernel that can handle multiple data types. CuPy allows this with the use of a type placeholder. The above `my_kernel` can be made type-generic as follows:

```
my_kernel_generic = cp.ElementwiseKernel(
   'T x, T y',
   'T z',
   'z = (x - y) * (x - y)',
   'my_kernel_generic')
```

If a type specifier is one character, T in this case, it is treated as a **type placeholder**. Same character in the kernel definition indicates the same type. More than one type placeholder can be used in a kernel definition. The actual type of these placeholders is determined by the actual argument type. The ElementwiseKernel class first checks the output arguments and then the input arguments to determine the actual type. If no output arguments are given on the kernel invocation, only the input arguments are used to determine the type.

```
my_kernel_generic = cp.ElementwiseKernel(
    'X x, Y y',
    'Z z',
    'z = (x - y) * (x - y)',
    'my_kernel_generic')
```

Skip to content #### check this

> **Note**
>
> This kernel requires the output argument to be explicitly specified, because the type Z cannot be automatically determined from the input arguments X and Y.

ReductionKernel

The second type of CuPy custom kernel is the reduction kernel, which is focused on kernels of the Map-Reduce type. The ReductionKernel class has four extra parts:

- Identity value: Initial value of the reduction
- Mapping expression: Pre-processes each element to be reduced
- Reduction expression: An operator to reduce the multiple mapped values. Two special variables, a and b, are used for this operand
- Post-mapping expression: Transforms the resulting reduced values. The special variable a is used as input. The output should be written to the output variable

Here is an example to compute L2 norm along specified axies:

```
>>> import cupy as cp
>>> # user-defined kernel
>>> l2norm_kernel = cp.ReductionKernel(
    'T x',  # input arg list
    'T y',  # output arg list
    'x * x',  # mapping
    'a + b',  # reduction
    'y = sqrt(a)',  # post-reduction mapping function
    '0',  # identity value
    'l2norm'  # kernel name
    )
>>> # allocating array
>>> x = cp.arange(10, dtype=np.float32).reshape(2, 5)
>>> x
array([[0., 1., 2., 3., 4.],
       [5., 6., 7., 8., 9.]], dtype=float32)
>>> # kernel launch
>>> l2norm_kernel(x, axis=1)
array([ 5.477226 , 15.9687195], dtype=float32)
```

RawKernel

The last is the RawKernel class, which is used to define kernels from raw CUDA/HIP XXXXXXX source.

Skip to content

. allows you to call the kernel with CUDA's cuLaunchKernel interface, and this gives you control of e.g. the grid size, block size, shared memory size, and stream.

```
>>> add_kernel = cp.RawKernel(r'''
... extern "C" __global__
... void my_add(const float* x1, const float* x2, float* y) {
...     int tid = blockDim.x * blockIdx.x + threadIdx.x;
...     y[tid] = x1[tid] + x2[tid];
... }
... ''', 'my_add')
>>> x1 = cp.arange(25, dtype=cp.float32).reshape(5, 5)
>>> x2 = cp.arange(25, dtype=cp.float32).reshape(5, 5)
>>> y = cp.zeros((5, 5), dtype=cp.float32)
>>> add_kernel((5,), (5,), (x1, x2, y))
>>> y
array([[ 0.,  2.,  4.,  6.,  8.],
       [10., 12., 14., 16., 18.],
       [20., 22., 24., 26., 28.],
       [30., 32., 34., 36., 38.],
       [40., 42., 44., 46., 48.]], dtype=float32)
```

> **Note**
>
> The kernel does not have return values. You need to pass both input arrays and output arrays as arguments.
>
> When using printf() in your GPU kernel, you may need to synchronize the stream to see the output.
>
> The kernel is declared in an extern “C” block, indicating that the C linkage is used. This is to ensure the kernel names are not mangled so that they can be retrieved by name.

cupy.fuse decorator

Apart from using the above templates for custom kernels, CuPy provides the cupy.fuse decorator which “fuse” the custom kernel functions to a single kernel function, therefore creating a dramatic lowering of the launching overhead. Moreover, the syntax looks like a Numba decorator, it is much easier to define an elementwise or reduction kernel than using the ElementwiseKernel or ReductionKernel template.

However, it is still experimental, i.e. there are bugs and incomplete functionalities to be fixed.

Here is the example using cupy.fuse() decorator

Skip to content

```
>>> import cupy as cp
>>> # adding decorator to the function squared_diff
>>> @cp.fuse(kernel_name='squared_diff')
... def squared_diff(x, y):
...     return (x - y) * (x - y)
>>> # allocating x and y
>>> x = cp.arange(10,dtype=cp.float32)
>>> y = x[::-1]
>>> # call the function
>>> squared_diff(x, y)
array([81, 49, 25,  9,  1,  1,  9, 25, 49, 81])
```

Low-level features

In addition to custom kernels, accessing low-level CUDA/HIP features are available for those who need more fine-grain control for performance:

- Stream and Event: CUDA stream and per-thread default stream are supported by all APIs
- Memory Pool: Customizable memory allocator with a built-in memory pool
- Profiler: Supports profiling code using CUDA Profiler and NVTX
- Host API Binding: Directly call CUDA libraries, such as NCCL, cuDNN, cuTENSOR, and cuSPARSELt APIs from Python

Summary

In this episode, we have learned about:

- CuPy basics
- Moving data between the CPU and GPU devices
- Different ways to launch GPU kernels

Keypoints

- GPUs have massive computing power compared to CPU
- CuPy is a good first step to start
- CuPy provides an extensive collection of GPU array functions
- Always have both the CPU and GPU versions of your code available so that you can compare performance, as well as validate the results
- Fine-tuning for optimal performance of real-world applications can be tedioius

Skip to content

See also

- CuPy Homepage
- GPU programming: When, Why and How?
- CUDA Python from Nvidia

Quick Reference

Instructor's guide

Why we teach this lesson

Intended learning outcomes

Timing

Preparing exercises

e.g. what to do the day before to set up common repositories.

Other practical aspects

Interesting questions you might get

Typical pitfalls

Learning outcomes

FIXME

This material is for …

By the end of this module, learners should:

- …
- …

Skip to content

See also

Credit

FIXME

Don’t forget to check out additional course materials from …

License

CC BY-SA for media and pedagogical material

Copyright © 2025 XXX. This material is released by XXX under the Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0).

Canonical URL: https://creativecommons.org/licenses/by-sa/4.0/

See the legal code

You are free to

1. **Share** — copy and redistribute the material in any medium or format for any purpose, even commercially.
2. **Adapt** — remix, transform, and build upon the material for any purpose, even commercially.
3. The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms

1. **Attribution** — You must give appropriate credit , provide a link to the license, and indicate if changes were made . You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
2. **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.
3. **No additional restrictions** — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Notices

You do not have to comply with the license for elements of the material in the public domain or where your use is permitted by an applicable exception or limitation .

No warranties are given. The license may not give you all of the permissions necessary for your intended use. For example, other rights such as publicity, privacy, or moral rights may limit how you use the material.

This deed highlights only some of the key features and terms of the actual license. It is not a license and has no legal value. You should carefully review all of the terms and conditions of the actual license before using the licensed material.

Skip to content

MIT for source code and code snippets

MIT License

Copyright (c) 2025, ENCCS project, The contributors

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.